Facultad de Ingeniería en Electricidad y Computación

Software Engineering II

Workshop #4

Acceptance Testing

Group 2

Aragón Intriago Shirley Yamel

Diaz Osorio Fernando Nahim

Muñoz Sánchez Salvador Gabriel

Navarrete Castillo Anthony Josué

Tumbaco Santana Gabriel Alejandro

July 6th, 2026

I PAO 2026

1. INTRODUCTION

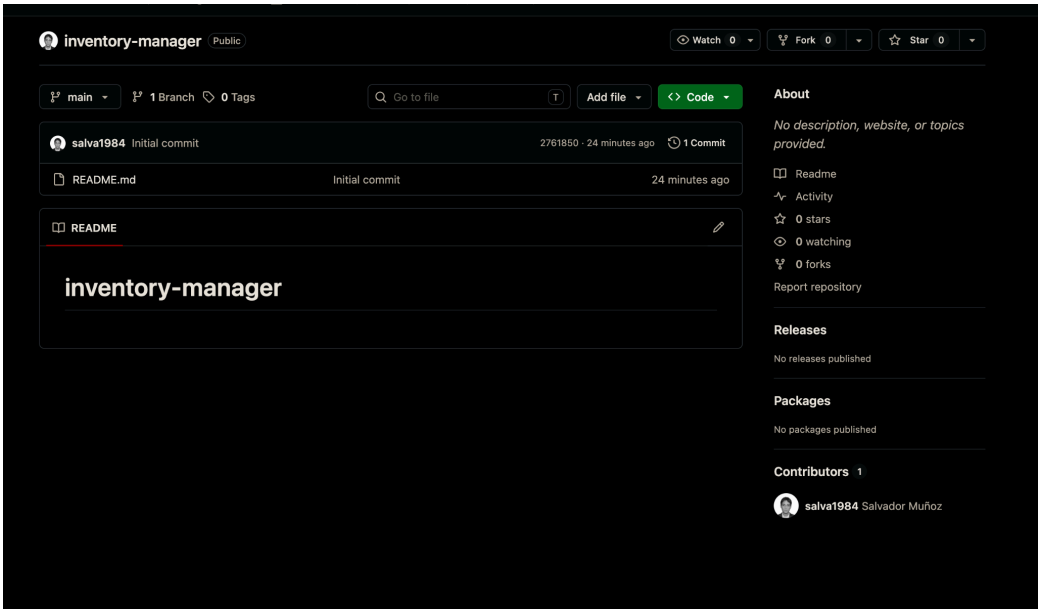
Behavior-Driven Development is a work methodology that seeks to reduce the communication gap between technical and non-technical members of a software team. This is achieved by creating a shared understanding of the problem to be solved, working in rapid iterations that increase the value stream, and producing living documentation that is automatically checked against the behavior of the system. Acceptance testing is a critical level of software testing where the system is evaluated against user needs, requirements, and business processes. Its fundamental purpose is to determine whether the system satisfies the acceptance criteria to enable its formal delivery. In this context, this report documents the development of a basic inventory management system in Python and the automation of its acceptance criteria using the Behave framework and the Gherkin language.

2. Development

2.1. Environment and Tools Configuration

For the development of this workshop, a Python 3.x environment was created inside a Github Repository, the steps used for this were as follows:

1. Initialization of the empty commit using Github
2. Local environment initialization and configuration within the Python environment, using these commands:
   python -m venv venv
   venv\Scripts\activate
3. Installation of Behave Library for the test development




```
@lolothens-e ➜ /workspaces/inventory-manager (main) $ python -m venv venv
@lolothens-e ➜ /workspaces/inventory-manager (main) $ pip install behave
Collecting behave
  Downloading behave-1.3.3-py2.py3-none-any.whl.metadata (10 kB)
Collecting cucumber-tag-expressions>=4.1.0 (from behave)
  Downloading cucumber_tag_expressions-10.0.0-py3-none-any.whl.metadata (4.7 kB)
Collecting cucumber-expressions>=17.1.0 (from behave)
  Downloading cucumber_expressions-20.0.0-py3-none-any.whl.metadata (1.7 kB)
Collecting parse>=1.18.0 (from behave)
  Downloading parse-1.22.1-py2.py3-none-any.whl.metadata (21 kB)
Collecting parse-type>=0.6.0 (from behave)
  Downloading parse_type-0.6.6-py2.py3-none-any.wh
Open folder in new window (cmd + click)
Requirement already satisfied: six>=1.15.0 in /hom                -packages (from behave) (1.17.0)
Requirement already satisfied: colorama>=0.3.7 in /home/codespace/.local/lib/python3.12/site-packages (from behave) (0.4.6)
Downloading behave-1.3.3-py2.py3-none-any.whl (223 kB)
Downloading cucumber_expressions-20.0.0-py3-none-any.whl (20 kB)
Downloading cucumber_tag_expressions-10.0.0-py3-none-any.whl (9.7 kB)
Downloading parse-1.22.1-py2.py3-none-any.whl (20 kB)
Downloading parse_type-0.6.6-py2.py3-none-any.whl (27 kB)
Installing collected packages: parse, parse-type, cucumber-tag-expressions, cucumber-expressions, behave
Successfully installed behave-1.3.3 cucumber-expressions-20.0.0 cucumber-tag-expressions-10.0.0 parse-1.22.1 parse-type-0.6.6

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python3 -m pip install --upgrade pip
```

2.2. App Code and Architecture

The developed app is an Inventory Manager using CLI. In order to accomplish this, every product must have at least 4 attributes:

1. **ID:** Unique identifier of a product
2. **Name:** Description of the product.
3. **Stock:** Amount of a said product within inventory
4. **Price:** Sale price for customer.

Besides fulfilling the original 4 requirements of the project, additionally we added the **Total Cost** functionality that adds up all the monetary value of the inventory and displays it.

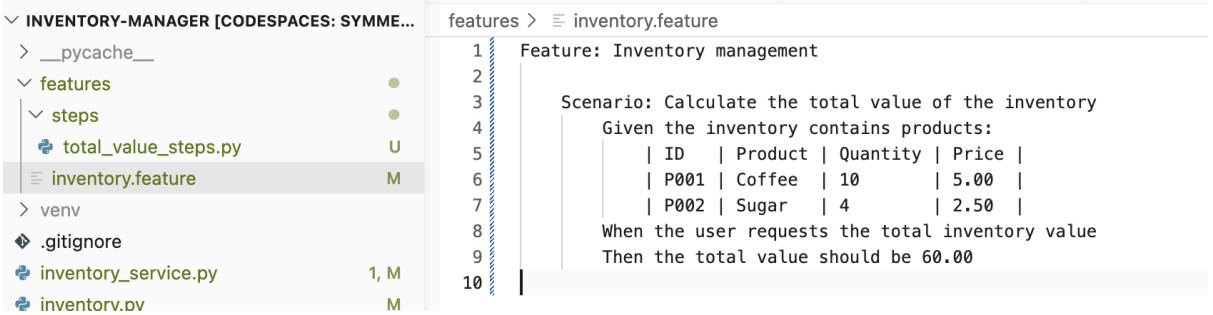
```
    elif args.command == "total":
        print(f"Total inventory value: {get_total_value(products):.2f}")
```

```
def get_total_value(products: list[dict[str, Any]]) -> float:
    return sum(float(product["quantity"]) * float(product["price"]) for product in products)
```

2.3. Characteristics and Scenarios Definition (Anthony Navarrete)

Using the suggested directory structure, we created the features/ folder containing an .feature file for the scenarios and another directory called steps/ where the tests were going to be developed. The file overall describes 5 scenarios, one thought after each functionality developed.

Here's the 5 scenarios written in Gherkin, as specified:



2.3.1 List all products in the inventory(Salvador Muñoz)

```
Scenario: List all products in the inventory
  Given the inventory contains products:
    | ID   | Product | Quantity | Price |
    | P001 | Coffee  | 10       | 5.00  |
    | P002 | Sugar   | 4        | 2.50  |
  When the user lists all products
  Then the output should contain:
    """
    Products:
    - Coffee
    - Sugar
    """
```

2.3.2 Add a product

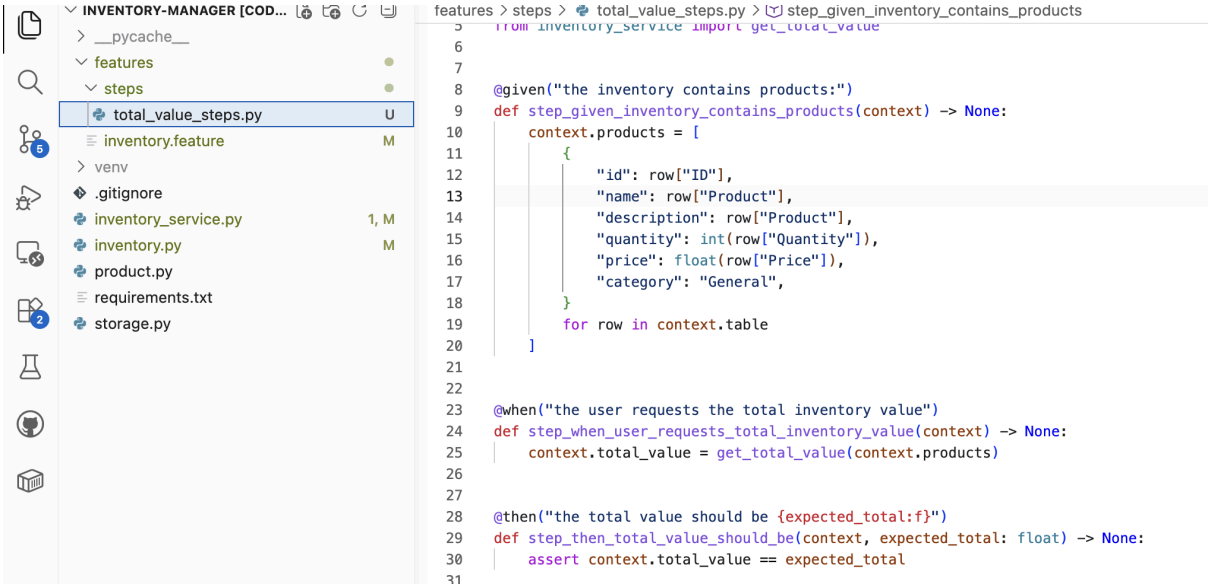
2.3.3 Update the quantity of a product (Gabriel Tumbaco)

```
15
16      Scenario: Update the quantity of a product
17          Given the inventory contains products:
18              | ID   | Product | Quantity | Price |
19              | P001 | Coffee  | 10       | 5.00  |
20          When the user updates product "P001" to quantity "25"
21          Then the inventory should show product "P001" with quantity "25"
22
```

2.3.4 Remove Product (Shirley Aragón)

2.4. Creation and Implementation of Acceptance Steps (Anthony Navarrete)

The steps described in the natural language specifications were mapped to executable code within the *features/steps/steps.py* directory. Priority was given to the Behave framework's guidelines by avoiding global variables and utilizing the context transfer object to isolate the state between scenarios.


INVENTORY-MANAGER [COD...
__pycache__
features
steps
total_value_steps.py U
inventory.feature M
venv
.gitignore
inventory_service.py 1, M
inventory.py M
product.py
requirements.txt
storage.py
features > steps > total_value_steps.py > step_given_inventory_contains_products
from inventory_service import get_total_value
@given("the inventory contains products:")
def step_given_inventory_contains_products(context) -> None:
context.products = [
{
"id": row["ID"],
"name": row["Product"],
"description": row["Product"],
"quantity": int(row["Quantity"]),
"price": float(row["Price"]),
"category": "General",
}
for row in context.table
]
@when("the user requests the total inventory value")
def step_when_user_requests_total_inventory_value(context) -> None:
context.total_value = get_total_value(context.products)
@then("the total value should be {expected_total:f}")
def step_then_total_value_should_be(context, expected_total: float) -> None:
assert context.total_value == expected_total

2.4.1 List Steps (Salvador Muñoz)

```python
from behave import when, then
from inventory_service import list_products


@when("the user lists all products")
def step_when_user_lists_all_products(context) -> None:
    """
    Step definition for Gherkin: 'When the user lists all products'.

    This step retrieves all products currently stored in the scenario's
    context using `list_products` from the inventory service, formats the
    list as a plain text string listing product names, and stores the formatted
    output in `context.output` for assertion in the 'Then' steps.
    """
    # Use the inventory service to list products
    products = list_products(context.products)
    lines = ["Products:"]
    for p in products:
        lines.append(f"- {p['name']}")
    context.output = "\n".join(lines)


@then("the output should contain:")
def step_then_output_should_contain(context) -> None:
    """
    Step definition for Gherkin: 'Then the output should contain:'.

    This step verifies that the multiline text block (docstring) specified
    in the Gherkin feature file is contained within the formatted output
    saved in `context.output`.
    """
    expected_output = context.text.strip()
    actual_output = context.output.strip()
    assert expected_output in actual_output, (
        f"Expected output to contain:\n{expected_output}\nbut was:\n{actual_output}"
    )
```

2.4.2 Update Quantity Steps (Gabriel Tumbaco)

```python

def update_product_quantity(products: list[dict[str, Any]], product_id: str, quantity: int) -> list[dict[str, Any]]:
    if quantity < 0:
        raise ValueError("Quantity cannot be negative")

    if find_product(products, product_id) is None:
        raise ValueError(f"Product '{product_id}' was not found")

    updated_products = deepcopy(products)
    for product in updated_products:
        if product["id"] == product_id:
            product["quantity"] = quantity
            break
    return updated_products

```

```python
from inventory_service import find_product, update_product_quantity


@when('the user updates product "{product_id}" to quantity "{quantity}"')
def step_when_user_updates_quantity(context, product_id, quantity):
    context.products = update_product_quantity(context.products, product_id, int(quantity))


@then('the inventory should show product "{product_id}" with quantity "{quantity}"')
def step_then_inventory_should_show_quantity(context, product_id, quantity):
    product = find_product(context.products, product_id)
    assert product is not None, f"Product '{product_id}' not found"
    assert product["quantity"] == int(quantity), (
        f"Expected quantity {quantity} but got {product['quantity']}"
    )

```

2.5. Test Execution and Iteration History (Anthony Navarrete)

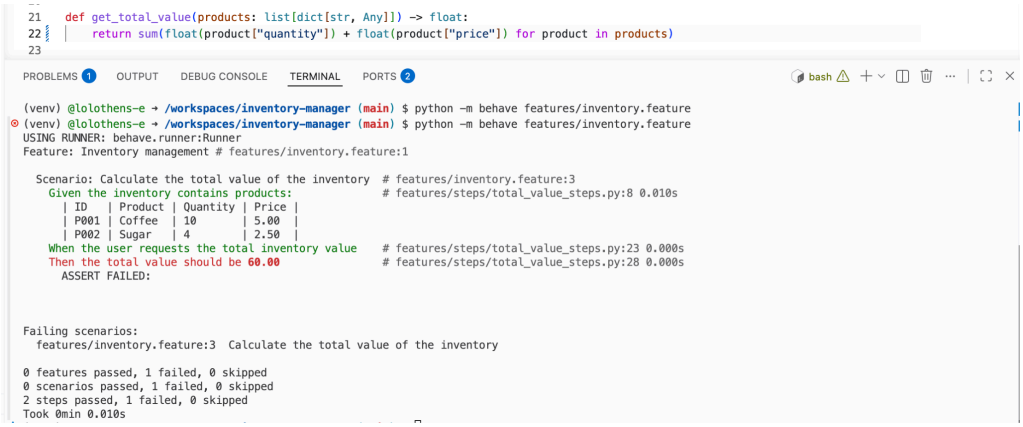
```
(venv) @lolothens-e → /workspaces/inventory-manager (main) $ python -m behave features/inventory.feature
USING RUNNER: behave.runner:Runner
Feature: Inventory management # features/inventory.feature:1

  Scenario: Calculate the total value of the inventory  # features/inventory.feature:3
    Given the inventory contains products:              # features/steps/total_value_steps.py:8 0.000s
      | ID   | Product | Quantity | Price |
      | P001 | Coffee  | 10       | 5.00  |
      | P002 | Sugar   | 4        | 2.50  |
    When the user requests the total inventory value    # features/steps/total_value_steps.py:23 0.000s
    Then the total value should be 60.00                # features/steps/total_value_steps.py:28 0.000s

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.000s
```

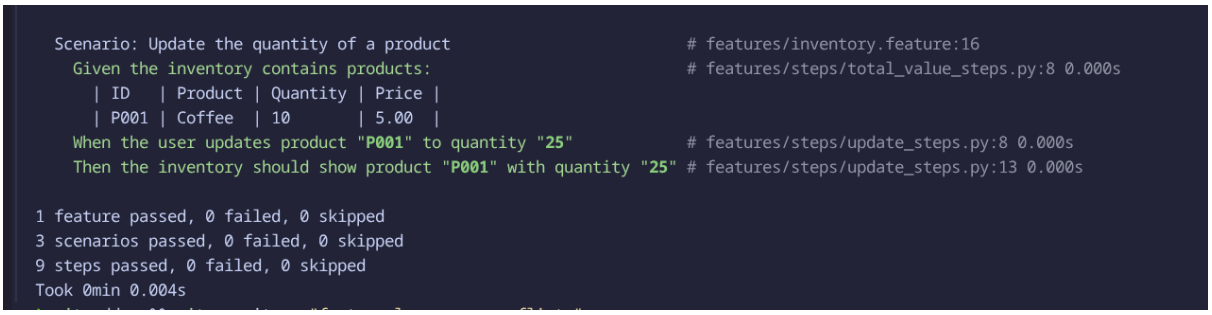
Due to the simplicity of the calculation within the additional functionality, the test passed on the first try, however, in order to demonstrate the functionality of the tests, the function was modified to cause an error.

After running the test with the modified function, we can notice that the test fails and



where exactly it fails, successfully demonstrating Behave and test implementation.

Output Gabriel Tumbaco



```
  Scenario: Update the quantity of a product                        # features/inventory.feature:16
    Given the inventory contains products:                          # features/steps/total_value_steps.py:8 0.000s
      | ID   | Product | Quantity | Price |
      | P001 | Coffee  | 10       | 5.00  |
    When the user updates product "P001" to quantity "25"           # features/steps/update_steps.py:8 0.000s
    Then the inventory should show product "P001" with quantity "25" # features/steps/update_steps.py:13 0.000s

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
9 steps passed, 0 failed, 0 skipped
Took 0min 0.004s
```

Output Salvador Muñoz

```
  Scenario: List all products in the inventory  # features/inventory.feature:23
    Given the inventory contains products:      # features/steps/total_value_steps.py:8 0.000s
      | ID   | Product | Quantity | Price |
      | P001 | Coffee  | 10       | 5.00  |
      | P002 | Sugar   | 4        | 2.50  |
    When the user lists all products            # features/steps/list_steps.py:7 0.000s
    Then the output should contain:             # features/steps/list_steps.py:25 0.000s
      """
      Products:
      - Coffee
      - Sugar
      """

1 feature passed, 0 failed, 0 skipped
4 scenarios passed, 0 failed, 0 skipped
12 steps passed, 0 failed, 0 skipped
Took 0min 0.003s
```

3. CONCLUSIONS AND RECOMMENDATIONS

Conclusions

- The acceptance tests developed with Behave successfully validated the implemented inventory management features, confirming that the application behaved according to the defined Gherkin scenarios.
- The use of Behavior Driven Development (BDD) helped describe the expected system behavior in a clear and understandable way, making it easier to relate the requirements to the Python implementation.

- Dividing the project into independent features allowed team members to work collaboratively while developing different functionalities of the same application.
- Executing the acceptance tests helped identify implementation issues during development, allowing the team to correct them before completing the project.

Recommendations

- Continue using acceptance tests whenever new features are added to ensure that the application continues to satisfy its expected behavior.
- Keep a consistent project structure and coding style across all project files to simplify collaboration and the integration of different features.
- Verify that all required functions and dependencies are implemented before executing the test suite to avoid errors caused by incomplete code.
- Run the Behave tests after each significant change to detect problems early and verify that the implemented functionality continues to work correctly.

4. REFERENCES

- **[1]** Software Testing Fundamentals, "Acceptance Testing," [Online]. Available: http://softwaretestingfundamentals.com/acceptance-testing/.
- **[2]** R. Hewitt, "Gherkin for Business Analysts," Modern analyst, 2017. [Online]. Available: https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/3810/Gherkin-for-Business-Analysts.aspx.
- **[3]** Cucumber, "Gherkin Reference - Keywords," [Online]. Available: https://cucumber.io/docs/gherkin/reference/.
- **[4]** Behave Documentation, "Tutorial: Feature Files & Step Definitions," [Online]. Available: https://behave.readthedocs.io/en/latest/tutorial/.
- **[5]** S. Vergara, "¿Qué es BDD (Behavior Driven Development)?," ITDO, Julio 2019. [Online]. Available: https://www.itdo.com/blog/que-es-bdd-behavior-driven-development/.

LINK TO REPOSITORY: https://github.com/salva1984/inventory-manager